

DevOps. Automatización y monitorización de infraestructuras

Unidad 06. Automatización y CI/CD: Despliegue y monitorización con Jenkins

- Laboratorio 01



Autor: Sergi García Barea

Actualizado Mayo 2026



Licencia



Reconocimiento – NoComercial – CompartirIgual (BY-NC-SA):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Unidad 06. Automatización y CI/CD: Despliegue y monitorización con Jenkins - Laboratorio 01	2
1. Explicación ¿Qué haremos en este caso práctico?	2
2. Objetivos	2
3. El modelo mental	2
4. Topología del proyecto	3
5. Fase de ejecución: despliegue paso a paso	5
Paso 1: Orquestación de red — El puente LabCEFIRE	5
Paso 2: Precarga de imágenes Docker	5
Paso 3: Infraestructura como Código — Terraform	6
Paso 4: Configuración de herramientas — Jenkins + Gitea	7
Paso 5: App v1 (Suma) — Primer pipeline CI/CD	9
Paso 6: App v2 (Multiplicación) — Upgrade de versión	10
Paso 7: Verificación de entornos (opcional)	11
Paso 8: Monitorización (opcional)	12
6. Pruebas manuales	12
6.1 El centro de mando: Grafana	12
6.2 El motor de métricas: Prometheus	13
6.3 El microscopio de Docker: cAdvisor	13
6.4 Verificación de la cadena CI/CD desde el navegador	14
7. Operaciones de limpieza	14
8. Problemas comunes y soluciones	15

UNIDAD 06. AUTOMATIZACIÓN Y CI/CD: DESPLIEGUE Y MONITORIZACIÓN CON JENKINS - LABORATORIO 01

1. EXPLICACIÓN ¿QUÉ HAREMOS EN ESTE CASO PRÁCTICO?

En este laboratorio vamos a construir un pipeline completo de CI/CD (Integración Continua y Despliegue Continuo) para automatizar el ciclo de vida de una aplicación web. Dejaremos atrás los despliegues manuales y configuraremos un ecosistema donde cada cambio en el código fuente desencadena automáticamente una cadena de eventos: pruebas unitarias, construcción de una imagen Docker, despliegue en un entorno de pruebas (staging), validación mediante smoke tests y —solo tras una aprobación humana— promoción a producción.

Utilizaremos Jenkins como orquestador central, Gitea como repositorio Git privado y Docker como plataforma de contenedores. Además, desplegamos un sistema de monitorización con Prometheus, cAdvisor y Grafana para visualizar en tiempo real la salud de los servicios y el rendimiento de los contenedores.

Al finalizar, tendrás una fábrica de software automatizada donde el código viaja desde el repositorio hasta los usuarios reales con controles de calidad y barreras de seguridad.

2. OBJETIVOS

Al terminar este caso práctico, habrás logrado:

- **Orquestación de red:** Crear un puente Docker aislado con direcciones IP estáticas para todos los servicios.
- **Infraestructura como Código:** Desplegar contenedores de forma declarativa con Terraform, incluyendo redes, volúmenes persistentes y configuraciones inyectadas.
- **Repositorio Git local:** Configurar Gitea como almacén de código fuente con autenticación básica.
- **Pipeline CI/CD:** Diseñar un pipeline Jenkins declarativo con etapas de checkout, tests unitarios, construcción de imagen, despliegue a staging, smoke test, aprobación manual y despliegue a producción.
- **Despliegue:** Separar entornos de pruebas y producción para validar versiones.
- **Monitorización:** Instalar y configurar Prometheus (recolección de métricas), cAdvisor (métricas de contenedores) y Grafana (visualización con dashboard automático).

3. EL MODELO MENTAL

Para entender cómo encajan las piezas de esta cadena de montaje de software, imagina que diriges una fábrica inteligente. Cada estación tiene un rol concreto:

El almacén de planos (Gitea)

Donde los ingenieros depositan los diseños (código fuente). Es un repositorio Git autogestionado que guarda cada versión, como un archivador en la oficina técnica.

El robot capataz (Jenkins)

El autómatas que lee los planos, organiza el trabajo y coordina toda la línea de producción. Cuando detecta un cambio en el almacén, lanza una secuencia de tareas sin intervención humana hasta que necesita aprobación. (Puerto 8080)

Las cajas de envío (Docker)

El formato estándar para empaquetar el producto. Cada versión del software se convierte en una imagen inmutable, como una caja de cartón con todo lo necesario dentro.

El taller de pruebas (Staging)

Una réplica idéntica del entorno de ventas, pero con acceso restringido. Aquí se enciende la nueva versión para que los probadores la inspeccionen antes de que llegue al público. (Contenedor en IP 172.28.0.62)

La sala de ventas (Producción)

El escaparate que ven los clientes reales. Solo se actualiza si todas las comprobaciones anteriores han sido satisfactorias y un responsable pulsa el botón de autorización. (Contenedor en IP 172.28.0.63)

El panel de control (Prometheus + Grafana)

Un conjunto de sensores y pantallas murales que muestran en tiempo real la salud de la fábrica. Cada máquina emite métricas que Prometheus recoge y Grafana convierte en gráficas de temperatura, ritmo de producción y tasas de error.

La línea de montaje sigue una secuencia rígida, como una cadena de producción con estaciones de verificación:

Nuevo plano en el almacén (Gitea) → El capataz (Jenkins) lo detecta automáticamente

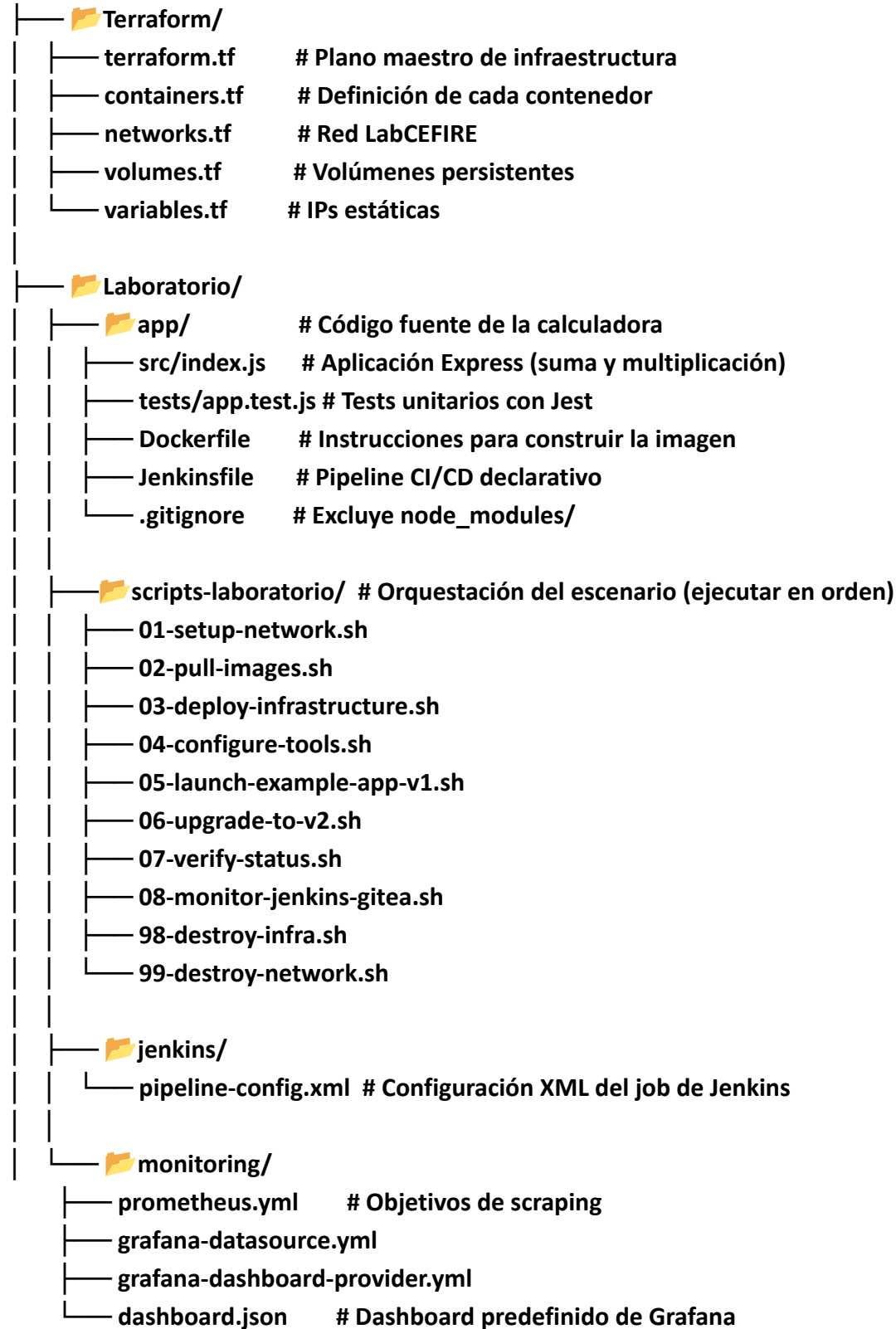
- 1. Recoge los planos (git checkout)
- 2. Control de calidad básico (tests unitarios con Jest)
- 3. Empaqueta el producto (docker build)
- 4. Lleva una muestra al taller de pruebas (despliegue en staging, IP 172.28.0.62)
- 5. Inspección rápida: ¿responde la máquina? (curl /health)
- 6.  Parada obligatoria: un operario revisa el prototipo
- 7. Si el operario da luz verde, se sustituye el producto en la sala de ventas (despliegue en producción, IP 172.28.0.63)

Si la inspección rápida falla (el endpoint /health no contesta correctamente), la cadena se detiene en seco y la producción no se toca. Solo la decisión humana tras verificar el taller de pruebas permite que la nueva versión llegue al cliente.

4. TOPOLOGÍA DEL PROYECTO

La infraestructura se basa en esta estructura de archivos, que separa claramente los planos de Terraform, el código de la aplicación, los scripts de orquestación y las configuraciones de monitorización:

📁 Unidad-06-CI-CD-Jenkins/



Nota sobre persistencia de datos:

- Jenkins usa el volumen jenkins-data para guardar jobs, credenciales y plugins.
- Gitea utiliza PostgreSQL (volumen postgres-data) para mejor rendimiento y fiabilidad.
- Ambos volúmenes persisten aunque destruyas los contenedores. El script 98-destroy-infra.sh los elimina para reinicios completamente limpios.

5. 🚀 FASE DE EJECUCIÓN: DESPLIEGUE PASO A PASO

A continuación desplegaremos todo el ecosistema. Los scripts están diseñados para ser idempotentes: puedes ejecutarlos múltiples veces y siempre obtendrás el mismo resultado.

Requisitos previos: Tener instalados Docker y Terraform en tu sistema anfitrión.

Abre una terminal y sitúate en la carpeta Laboratorio/scripts-laboratorio/.

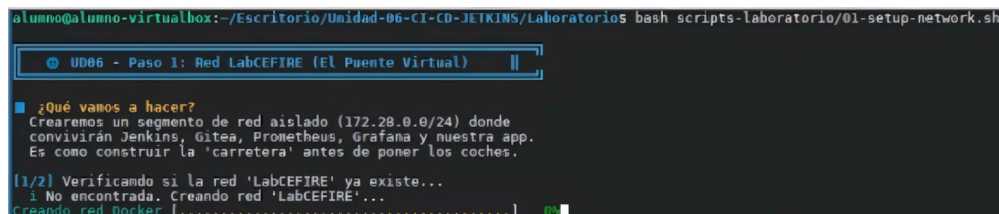
Paso 1: 🌐 Orquestación de red — El puente LabCEFIRE

Crea una red Docker de tipo bridge llamada LabCEFIRE con la subred 172.28.0.0/24. Es el cimiento del laboratorio: todos los contenedores se conectarán a esta red y recibirán IPs estáticas, garantizando que siempre sepamos dónde está cada servicio.

Ejecución:

```
cd Laboratorio/scripts-laboratorio
bash 01-setup-network.sh
```

¿Qué ocurre bajo el capó?



El script habla directamente con el subsistema de red del kernel de Linux de tu equipo anfitrión:

- **ip link add name LabCEFIRE type bridge** → Crea un conmutador (switch) lógico de capa 2.
- **ip addr add 172.28.0.1/24 dev LabCEFIRE** → Asigna una dirección IP a la interfaz virtual de tu PC, que actuará como puerta de enlace (gateway) para todos los contenedores.

📌 Nota didáctica:

Una red bridge en Docker actúa como un switch virtual. Al definir la subred manualmente, aseguramos que las IPs sean predecibles. Así, Prometheus y Jenkins pueden usar direcciones fijas sin depender de un servidor DNS.

Paso 2: 📦 Precarga de imágenes Docker

Qué hace:

Descarga todas las imágenes necesarias (Jenkins, Gitea, PostgreSQL, Node, Prometheus, Grafana, cAdvisor) antes de desplegar los contenedores. Esto asegura que el laboratorio funcione sin depender de Internet durante la ejecución de Terraform y evita tiempos de espera.

Ejecución:

```
bash 02-pull-images.sh
```

¿Qué ocurre bajo el capó?

```
alumno@alumno-virtualbox: ~/Escritorio/Unidad-06-CI-CD-JETKINS/Laboratorio$ bash scripts-laboratorio/02-pull-images.sh
UD06 - Paso 2: Precarga de imágenes Docker
¿Qué vamos a hacer?
Descargaremos todas las imágenes necesarias para el laboratorio
para que el despliegue con Terraform sea más rápido y funcione
sin depender de Internet durante la ejecución.
[PULL] [1/7] jenkins/jenkins:lts-jdk21 - ya en caché
[PULL] [2/7] gitea/gitea:1.17 - ya en caché
[PULL] [3/7] postgres:16-alpine - ya en caché
[PULL] [4/7] node:20-alpine - ya en caché
[PULL] [5/7] prom/prometheus:latest - ya en caché
```

El script itera sobre una lista predefinida de imágenes y ejecuta docker pull para cada una. Si alguna imagen ya existe localmente, Docker la omite.



La explicación del arquitecto:

Precargar imágenes evita que terraform apply se demore descargándolas en el momento del despliegue. Es una práctica recomendada en entornos de laboratorio y también en producción cuando se usan registros privados lentos o se trabaja sin conexión a Internet.

Paso 3: 🏗️ Infraestructura como Código — Terraform

Qué hace:

Inicializa Terraform y despliega los contenedores base del laboratorio (los servicios de soporte). Las IPs de staging y producción quedan reservadas, pero los contenedores de la aplicación los creará el pipeline de Jenkins más adelante.

Los contenedores desplegados son:

Contenedor	IP	Puerto expuesto	Imagen
ud06-postgres	172.28.0.68	5432	postgres:16-alpine
ud06-gitea	172.28.0.64	3000	gitea/gitea:1.21
ud06-jenkins	172.28.0.61	8080, 50000	ud06-jenkins-custom (build local)
ud06-prometheus	172.28.0.65	9090	prom/prometheus
ud06-grafana	172.28.0.66	3000	grafana/grafana
ud06-cadvisor	172.28.0.67	8080	gcr.io/cadvisor/cadvisor

Ejecución:

```
bash 03-deploy-infrastructure.sh
```

¿Qué ocurre bajo el capó?

```
total orientativo: 6-15 minutos
i Los contadores en vivo muestran el progreso.
i Si un servicio excede el máximo, se muestran sus
  últimas líneas de log y se continúa con el siguiente.

[1/7] Validando entorno...
✓ Entorno validado
[2/7] Preparando directorios de la aplicación y datos...
✓ Directorios listos
[3/7] Precargando imagen para tests (node:20-alpine)...
[DEPLOY] node:20-alpine ya en caché
✓ Imagen de tests precargada
[4/7] Sincronizando estado de Terraform...
✓ Estado sincronizado
[5/7] Limpiando conflictos en Docker...
✓ Sin conflictos
[6/7] Desplegando infraestructura con Terraform...
[DEPLOY] Inicializando Terraform...
✓ Terraform inicializado
[DEPLOY] Aplicando plan de Terraform...
△ Puede tardar hasta 5 minutos. Lo más lento:
1. Descargar imagen base jenkins/jenkins:lts-jdk21 (~400MB)
2. Construir imagen personalizada (instalar Docker CLI)
3. Descargar postgres:16-alpine, prom/prometheus, etc.
i Si todo está en caché (02-pull-images.sh), esto vuela.

Terraform ejecutándose... [8s]
```

El script actúa como un envoltorio de seguridad:

1. Limpia contenedores huérfanos y conflictos de nombres (idempotencia).
2. Ejecuta terraform init (descarga el proveedor Docker).
3. Ejecuta terraform apply -auto-approve (traduce el código HCL en llamadas a la API de Docker).
4. Inyecta archivos de configuración mediante bind mounts:
 - pipeline-config.xml → /var/jenkins_home/jobs/Calculator-Pipeline/config.xml
 - prometheus.yml → /etc/prometheus/prometheus.yml
 - Archivos de provisioning de Grafana → /etc/grafana/provisioning/

Nota didáctica – Terraform:

En lugar de escribir docker run manualmente, describimos el estado deseado en archivos .tf. Terraform calcula las diferencias y aplica solo los cambios necesarios. Esto se llama idempotencia: ejecutar terraform apply 100 veces produce el mismo resultado. El archivo terraform.tfstate guarda la memoria de lo que ya existe.

Paso 4: Configuración de herramientas — Jenkins + Gitea

Qué hace:

Configura Jenkins (plugins, credenciales, job) y Gitea (usuario, repositorio) de forma totalmente automatizada, sin intervención manual en las interfaces web.

Ejecución:

```
bash 04-configure-tools.sh
```


¿Qué ocurre bajo el capó? (Secuencia de 6 pasos)

```

UD06 - Paso 4: Configuración de Jenkins y Gitea

■ ¿Qué vamos a hacer?
Configuraremos Jenkins (plugins, job pipeline, credenciales
para Gitea) y Gitea (usuario gitea-admin y repositorio
calculator-app). Así dejamos listo el ecosistema para
recibir el código de la aplicación.

● Tiempos estimados:
▲ Jenkins inicial: hasta 4 min (esperar que arranque plugins)
▲ Plugins + Job: ~20 s (instalar 4 plugins)
▲ Gitea configuración: ~10 s (crear usuario + repo)
▲ Reinicio Jenkins: hasta 3 min (cargar plugins + credenciales)

Total orientativo: 3-5 minutos
i Los contadores en vivo muestran el progreso.

[1/6] Esperando a que Jenkins esté listo...
▲ Jenkins puede tardar hasta 4 min descargando plugins.
URL: http://172.28.0.61:8080/login
✓ Jenkins listo (1s)
[2/6] Instalando plugins de Jenkins...

```

1. **Espera activa:** El script sondea el endpoint de Jenkins hasta que responde (máximo 4 minutos).
2. **Instalación de plugins:** Ejecuta `jenkins-plugin-cli --plugins git workflow-aggregator docker-workflow credentials-binding prometheus` dentro del contenedor.
3. **Creación del job:** Copia `pipeline-config.xml` al archivo llamado `/var/jenkins_home/jobs/Calculator-Pipeline/config.xml`. Este archivo define un pipeline que apunta al repositorio Gitea y usa polling SCM cada 2 minutos.
4. **Inyección de credenciales:** Un script Groovy (colocado en `init.groovy.d/`) crea la credencial gitea-admin en Jenkins.
5. **Configuración de Gitea:** Mediante llamadas a la API REST, crea el usuario gitea-admin y el repositorio `calculator-app`.
6. **Reinicio de Jenkins:** Se reinicia el contenedor para que cargue los plugins y las credenciales (espera hasta 3 minutos).

Verificación (navegador):

Recurso	URL	Qué esperar
Jenkins Pipeline	http://172.28.0.61:8080/job/Calculator-Pipeline/	Job configurado (sin builds aún)
Gitea Repositorio	http://172.28.0.64:3000/gitea-admin/calculator-app	Repositorio vacío

🔑 Credenciales del laboratorio:

Servicio	URL	Usuario	Contraseña
Jenkins	http://172.28.0.61:8080	admin	admin

Gitea	http://172.28.0.64:3000	gitea-admin	admin
Grafana	http://172.28.0.66:3000	admin	admin

💡 La explicación del arquitecto – Pipeline como código:

El job de Jenkins no tiene una configuración manual de los pasos del pipeline. En su lugar, apunta al repositorio Gitea y busca un archivo **Jenkinsfile** en la raíz. El pipeline se define dentro del código fuente, viaja con la aplicación y se versiona junto a ella. Esto es Pipeline as Code, una de las mejores prácticas de CI/CD.

Paso 5: 🚀 App v1 (Suma) — Primer pipeline CI/CD

Qué hace:

Crea la primera versión de la calculadora (suma, interfaz azul), sube el código a Gitea y dispara el pipeline de Jenkins. El pipeline se pausará en el stage de aprobación manual.

Ejecución:

```
bash 05-launch-example-app-v1.sh
```

¿Qué ocurre bajo el capó?

1. Escribe src/index.js con la lógica de suma y tests/app.test.js correspondiente.
2. Inicializa un repositorio Git y hace git push a Gitea.
3. Obtiene un crumb CSRF de Jenkins y lanza el build mediante curl -X POST.
4. Sigue el progreso del build mostrando la consola en vivo.

El pipeline (**Jenkinsfile**) ejecuta estos stages:

Stage	Acción
Checkout	Clona el repositorio desde Gitea
Unit Tests	npm install && npm test dentro de un contenedor Node 20
Build Image	docker build -t calculator-app:latest .

Deploy Staging	Lanza contenedor ud06-staging en IP 172.28.0.62
Smoke Test Staging	Espera 10 segundos y hace curl /health → debe devolver "healthy"
Aprobar despliegue	PAUSA – espera clic en "Proceed" desde la interfaz de Jenkins
Deploy Production	(Solo tras aprobar) Lanza contenedor ud06-production en IP 172.28.0.63

Verificación (navegador):

Recurso	URL	Resultado esperado
Pipeline	http://172.28.0.61:8080/job/Calculator-Pipeline/	Build #1 en progreso o completado hasta el stage de aprobación
App Staging	http://172.28.0.62:3000/	Calculadora azul (suma)
Health Staging	http://172.28.0.62:3000/health	{"status":"healthy","version":"1.0.0"}
App Producción	http://172.28.0.63:3000/	"No se puede conectar" (aún no desplegado)

Nota importante:

El pipeline se detiene en el stage Aprobar despliegue. Para que la v1 llegue a producción, debes ir a Jenkins, abrir el build y hacer clic en Proceed. Mientras no lo hagas, producción permanece vacía (o con la versión anterior).

Paso 6: App v2 (Multiplicación) — Upgrade de versión

Qué hace:

Actualiza la calculadora a la versión 2 (multiplicación, interfaz verde), sube el cambio a Gitea y dispara un nuevo pipeline. Esta vez, staging se actualizará automáticamente, pero producción seguirá en v1 hasta que apruebes manualmente.

Ejecución:

```
bash 06-upgrade-to-v2.sh
```

```

Vamos a evolucionar la calculadora de la v1 (suma, azul) a la
v2 (multiplicaci\u00f3n, verde). El c\u00f3digo se subir\u00e1 a Gitea,
Jenkins detectar\u00e1 el cambio, ejecutar\u00e1 el pipeline completo,
y se pausar\u00e1 esperando aprobaci\u00f3n manual para Producci\u00f3n.

[1/4] Escribiendo c\u00f3digo de la app v2 (Multiplicaci\u00f3n)...
  / C\u00f3digo v2 escrito
[2/4] Subiendo v2 a Gitea...
M   src/index.js
M   tests/app.test.js
Tu rama est\u00e1 actualizada con 'origin/main'.
  / C\u00f3digo v2 subido a Gitea (rama main)
  / Push completado
[3/4] Disparando pipeline en Jenkins...
  / \u00d9ltimo build antes del trigger: #2
  / Obteniendo crumb CSRF
  / Build disparado (HTTP 201)
[4/4] Siguiendo el progreso del build...
x El pipeline tarda entre 3 y 5 minutos
(npm install + docker build + smoke test)

Esperando build [9s / 120s] / Build #3 iniciado
En ejecuci\u00f3n: 8s / 900s | [IN_PROGRESS] Declarative: Checkout SCM
En ejecuci\u00f3n: 3s / 900s | [SUCCESS] Declarative: Checkout SCM
En ejecuci\u00f3n: 10s / 900s | [IN_PROGRESS] Checkout
En ejecuci\u00f3n: 15s / 900s | [IN_PROGRESS] Unit Tests
En ejecuci\u00f3n: 46s / 900s

```

¿Qué cambia respecto a v1?

Aspecto	v1 (Suma)	v2 (Multiplicaci\u00f3n)
Operaci\u00f3n	+	×
Color de fondo	Azul (#1976d2)	Verde (#2e7d32)
Versi\u00f3n en /health	1.0.0	2.0.0
Test unitario	5 + 5 = 10	5 × 5 = 25

Verificaci\u00f3n tras el script:

```

curl http://172.28.0.62:3000/health # {"status":"healthy","version":"2.0.0"}
curl http://172.28.0.63:3000/health # {"status":"healthy","version":"1.0.0"}

```

Explicaci\u00f3n del flujo:

- Staging (.62) se actualiza inmediatamente a la v2 (verde).
- Producci\u00f3n (.63) sigue en v1 (azul) hasta que apruebes.
- El nuevo build de Jenkins se pausa de nuevo en el stage de aprobaci\u00f3n.

⚠ Prueba antes de aprobar:

Abre en el navegador <http://172.28.0.62:3000> y verifica que la multiplicaci\u00f3n funciona correctamente. Si todo est\u00e1 bien, ve a Jenkins y haz clic en Proceed para desplegar v2 a producci\u00f3n.

Paso 7: 🔍 Verificaci\u00f3n de entornos (opcional)

Qu\u00e9 hace:

Consulta las aplicaciones en Staging y Producci\u00f3n, extrae la versi\u00f3n y el color del HTML, y muestra si ambos entornos est\u00e1n sincronizados o desfasados. \u00cdtil para saber r\u00e1pidamente en qu\u00e9 estado est\u00e1 el despliegue.

Ejecuci\u00f3n:

```
bash 07-verify-status.sh
```

```
cada cambio del repositorio. Es tu "zona de pruebas".
Producción es el entorno que ven los usuarios reales.
Solo se actualiza cuando tú lo apruebas manualmente.

Este flujo (staging → aprobar → producción) se llama
"deployment gate" y es una práctica recomendada en CI/CD.
Evita que código no verificado llegue a los usuarios.

# Credenciales del laboratorio:
Jenkins: admin / admin
Gitea: gitea-admin / admin
Grafana: admin / admin

# Contenedores activos:
ud06-production Up 21 seconds (healthy) 3000/tcp
ud06-staging Up About a minute (healthy) 3000/tcp
ud06-jenkins Up 13 minutes 8080/tcp, 50000/tcp
ud06-gitea Up 21 minutes 22/tcp, 3000/tcp

# PRÓXIMOS PASOS:
• Si ves diferencias → Ve a Jenkins http://172.28.0.61:8080
  y pulsa "Proceed" para desplegar a producción.
• Para ver el estado detallado → bash 08-monitor-jenkins-gitea.sh
• Para ver métricas → Grafana: http://172.28.0.66:3000 (admin/admin)
• Para ver métricas → Prometheus: http://172.28.0.65:9090
```

💡 La explicación del arquitecto:

El script descarga el HTML de cada entorno y busca dos indicadores: la cadena Versión X.Y.Z y el código de color de fondo (e3f2fd = azul v1, e8f5e9 = verde v2). Así podemos automatizar la auditoría visual sin depender de APIs específicas.

Paso 8: 📊 Monitorización (opcional)

Qué hace:

Consulta las APIs REST de Gitea y Jenkins para mostrar en una sola pantalla el estado actual del repositorio, el último commit y el estado del pipeline (ejecutando, pausado, éxito o fallo).

Ejecución:

```
bash 08-monitor-jenkins-gitea.sh
```

6. 🧑‍🔧 PRUEBAS MANUALES

Un ingeniero DevOps nunca asume que un despliegue ha sido exitoso solo porque los scripts no dieron errores. La infraestructura debe ser auditada y las métricas deben ser visibles. A continuación se describen las comprobaciones manuales que confirmarán que todo el ecosistema funciona correctamente.

6.1 El centro de mando: Grafana

Grafana es el videowall de nuestra fábrica. Muestra gráficos en tiempo real con la salud de los contenedores y la versión desplegada en cada entorno.

Acceso:

- URL: <http://172.28.0.66:3000>
- Usuario: admin
- Contraseña: admin

Verificación:

1. Inicia sesión con las credenciales.
2. En el menú lateral izquierdo, haz clic en "Dashboards" (cuatro cuadrados) y luego en "Browse".
3. Busca el dashboard llamado "UD06 - Centro de Mando DevOps" y ábrelo.
4. Debes ver varios paneles:

- Versión en Staging y Versión en Producción: dos grandes indicadores que muestran "v1.0.0" o "v2.0.0".
- Total de builds y Builds fallidos (datos provenientes de Jenkins).
- CPU (%) y RAM de cada contenedor (Jenkins, Gitea, Staging, Producción, Prometheus, Grafana, cAdvisor).
- Repositorios y Usuarios de Gitea.

Si ves "No data" en algún panel:

Espera 1-2 minutos a que Prometheus complete su primer ciclo de scraping (cada 15 segundos). Si persiste, revisa en Prometheus que los targets estén "UP" (ver sección 6.3).

6.2 El motor de métricas: Prometheus

Prometheus es el recolector de datos. Puedes inspeccionar directamente qué objetivos está raspando y si responden correctamente.

Acceso:

- URL: <http://172.28.0.65:9090>

Verificación:

1. Ve al menú superior Status → Targets.
2. Debes ver una lista de objetivos con su estado:
 - ud06-app-staging (172.28.0.62:3000) → UP
 - ud06-app-production (172.28.0.63:3000) → UP
 - ud06-cadvisor (172.28.0.67:8080) → UP
 - ud06-jenkins (172.28.0.61:8080) → UP
3. Si algún objetivo aparece en DOWN, revisa que el contenedor correspondiente esté funcionando:
4. bash
5. docker ps | grep ud06

Consulta de métricas:

En la pestaña Graph, prueba alguna consulta PromQL:

- app_version → muestra la versión de la calculadora en staging y producción.
- container_cpu_usage_seconds_total{name=~"ud06.*"} → uso de CPU por contenedor.
- default_jenkins_builds_success_build_count_total → builds exitosos de Jenkins.

6.3 El microscopio de Docker: cAdvisor

cAdvisor expone métricas profundas a nivel de kernel sobre cada contenedor. Útil para diagnósticos de rendimiento.

Acceso:

- URL: <http://172.28.0.67:8080>

Verificación:

1. Navega por la interfaz. Verás una lista de todos los contenedores del sistema (no solo los

del laboratorio).

2. Haz clic en ud06-staging o ud06-production para ver gráficas detalladas de CPU, memoria, red y sistema de archivos.
3. Comprueba que los datos se actualizan cada segundo.

6.4 Verificación de la cadena CI/CD desde el navegador

Además de las métricas, puedes seguir visualmente el pipeline:

Jenkins:

- URL: <http://172.28.0.61:8080/job/Calculator-Pipeline/>
- Abre el último build. Verás los stages en color:
 - Azul = ejecutado con éxito.
 - Rojo = fallido.
 - Gris = pendiente o no ejecutado (por ejemplo, producción cuando el pipeline está pausado).

Gitea:

- URL: <http://172.28.0.64:3000/gitea-admin/calculator-app>
- Verifica que los commits (v1 y v2) están en el historial.

7. OPERACIONES DE LIMPIEZA

Para devolver el sistema anfitrión a su estado original (por ejemplo, antes de repetir el laboratorio desde cero o para liberar recursos), usa los scripts de destrucción incluidos.

```
bash 98-destroy-infra.sh
```

¿Qué hace este script?

- Ejecuta **terraform destroy** para eliminar ordenadamente los contenedores gestionados por Terraform.
- Fuerza la eliminación de cualquier contenedor residual (incluyendo ud06-staging y ud06-production creados por Jenkins).
- Elimina los volúmenes jenkins-data y postgres-data (esto borra pipelines antiguos y la base de datos de Gitea).
- Borra imágenes locales construidas (calculator-app, ud06-jenkins-custom).
- Limpia el estado de Terraform (archivos .tfstate y directorio .terraform/).
- Finalmente, elimina la red LabCEFIRE.

Nota: Las imágenes descargadas de Docker Hub (Gitea, Jenkins, Prometheus, Grafana, cAdvisor, Node, PostgreSQL) se conservan para reutilización offline, ahorrando tiempo en futuros despliegues.

Si ya has eliminado los contenedores manualmente y solo quieres borrar la red virtual:

```
bash 99-destroy-network.sh
```

Este script baja y elimina la interfaz LabCEFIRE del kernel de tu sistema.

8. PROBLEMAS COMUNES Y SOLUCIONES

En una fábrica automatizada, los incidentes son inevitables. Estos son los fallos más frecuentes que hemos detectado durante las pruebas y cómo resolverlos para que la cadena de montaje no se detenga.

El panel mural (Grafana) se queda en negro y muestra "No data"

- Causa probable: El sensor Prometheus no está enviando información o no está bien conectado al panel.
- Solución:
 1. Entra en la sala de sensores: `http://172.28.0.65:9090/targets`. Si los objetivos aparecen DOWN, los contenedores no emiten métricas.
 2. En el panel de control de Grafana, ve a Configuration → Data Sources → Prometheus y pulsa Save & Test para comprobar la conexión.
 3. Si falla, reinicia el panel: `docker restart ud06-grafana`.

El tablero predefinido (dashboard) no aparece en Grafana

- Causa probable: El archivo dashboard.json no se montó correctamente en el panel.
- Solución:
 1. Verifica que el enlace físico existe:
`docker inspect ud06-grafana --format '{{range .Mounts}}{{.Source}} → {{.Destination}}{{"\n"}}{{end}}'`
Debe apuntar a la ruta de dashboard.json.
 2. Si no aparece, ejecuta de nuevo bash `03-deploy-infrastructure.sh`.

El robot capataz (Jenkins) detiene la cadena en "Smoke Test Staging"

- Causa probable: La máquina del taller de pruebas (staging, IP 172.28.0.62) no arrancó o su chequeo de salud devuelve algo distinto a healthy.
- Solución:
 1. Prueba manualmente: `curl http://172.28.0.62:3000/health`.
 2. Lee los últimos registros del contenedor de staging: `docker logs ud06-staging --tail 30`.
 3. Si el contenedor ni siquiera se creó, inspecciona el paso Deploy Staging en la consola del capataz.

El botón "Proceed" (luz verde de aprobación) no aparece en la interfaz de Jenkins

- Causa probable: La etapa de espera está activa, pero la pantalla del supervisor no se actualizó.
- Solución:

Recarga la página del build en Jenkins (F5). Si sigue sin aparecer, cancela la ejecución y dispara una nueva.

Jenkins se queja de que falta el plugin docker-workflow

- Causa probable: La instalación automática de complementos falló sin avisar.

- **Solución:**

1. Entra en la cabina del capataz y reinstálalo:
docker exec -it ud06-jenkins jenkins-plugin-cli --plugins docker-workflow.
2. Después reinicia: `docker restart ud06-jenkins.`

 Desde el puesto de administración (host) no se puede hacer curl a las IPs de la fábrica

- Causa probable: La carretera privada LabCEFIRE no tiene una ruta de vuelta hacia el host.
- **Solución:**
 1. Comprueba que el host posee la IP 172.28.0.1 en la interfaz LabCEFIRE: `ip addr show LabCEFIRE.`
 2. Si falta, vuelve a tender la red: `bash 01-setup-network.sh.`

 Las gráficas de CPU del panel están vacías, pero las de RAM sí funcionan

- Causa probable: El sensor de contenedores (cAdvisor) no está enviando métricas de CPU o la consulta PromQL en el panel es incorrecta.
- **Solución:**
 1. En Prometheus (<http://172.28.0.65:9090>) busca la métrica **`container_cpu_usage_seconds_total`**.
 2. Si no existe, verifica que cAdvisor está vivo: ***docker logs ud06-cadvisor --tail 20.***